**Program A:** ChatGPT
**Program B:** Gemini

**ChatGPT Accessed with:** ChatGPT Website
**Gemini Accessed with:** Google Search Query

# Prompt:

Generate C++ code using objects, not functions, to implement and build a program will prioritize emails for a busy company CEO.

Use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.

Must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.

Must create objects from scratch. Do not include pre-existing heap modules.

Test File format:

| Command | Description |
|---|---|
| EMAIL <sender category>, <subject line>, <date> | The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:<br>● Boss – read first<br>● Subordinate – read next<br>● Peer – read next<br>● ImportantPerson – read next<br>● OtherPerson – read last<br>If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.<br><br>EMAIL is followed by space. The rest of the fields are delimited.<br>Assume <sender category> is one of the five strings listed above.<br>Assume <subject line> is a string which may contain spaces, but not commas |

| | |
|---|---|
| | Assume <date> is in the format:<br>MM-DD-YYYY |
| NEXT | Next email for the CEO to read. Display the information on the terminal in the following format:<br>Sender:<br>Subject:<br>Date: |
| READ | CEO has read email and dealt with it |
| COUNT | Display current count of unread emails |

Your code will be tested with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

Sample Test File:

```
EMAIL Peer, Can you help me on this?, 12-01-2024
EMAIL OtherPerson, Try our product, 12-19-2024
EMAIL Boss, Important, 12-20-2024
EMAIL Subordinate, How do I handle this?,12-25-2024
EMAIL ImportantPerson, Health Insurance Enrollment, 12-31-2024
EMAIL Boss, Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

Your program should display this:

```
There are 6 emails to read.
```

```
Next email:
        Sender: Boss
        Subject: Never Mind
        Date: 01-03-2025

Next email:
        Sender: Boss
        Subject: Important
        Date: 12-20-2024

There are 4 emails to read.
```

# ChatGPT Generated Code:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <list>

using namespace std;


// ============================================================
// EMAIL OBJECT
// ============================================================

class Email {
private:
    string sender;
    string subject;
    string date;

public:
    Email() {
        sender = "";
        subject = "";
        date = "";
    }
```

```
Email(string senderCategory, string subjectLine, string emailDate) {
    sender = senderCategory;
    subject = subjectLine;
    date = emailDate;
}

string getSender() const {
    return sender;
}

string getSubject() const {
    return subject;
}

string getDate() const {
    return date;
}

// Convert sender category into a priority number
int getSenderPriority() const {
    if (sender == "Boss")
        return 5;
    else if (sender == "Subordinate")
        return 4;
    else if (sender == "Peer")
        return 3;
    else if (sender == "ImportantPerson")
        return 2;
    else
        return 1; // OtherPerson
}

// Convert MM-DD-YYYY into YYYYMMDD so dates can be compared
int getDatePriority() const {
    int month = stoi(date.substr(0, 2));
    int day = stoi(date.substr(3, 2));
    int year = stoi(date.substr(6, 4));

    return year * 10000 + month * 100 + day;
```

```
    }

    // Determine whether this email has higher priority
    bool hasHigherPriorityThan(const Email& other) const {

        // First compare sender category
        if (getSenderPriority() != other.getSenderPriority()) {
            return getSenderPriority() > other.getSenderPriority();
        }

        // If sender category is the same, newest date comes first
        return getDatePriority() > other.getDatePriority();
    }

    void display() const {
        cout << "Sender: " << sender << endl;
        cout << "Subject: " << subject << endl;
        cout << "Date: " << date << endl;
    }
};


// ============================================================
// MAX HEAP OBJECT
// ============================================================

class MaxHeap {
private:
    list<Email> heap;

    // Get an email at a particular index in the list
    list<Email>::iterator getIterator(int index) {
        list<Email>::iterator it = heap.begin();

        for (int i = 0; i < index; i++) {
            ++it;
        }

        return it;
    }
```

```
// Swap two elements in the list
void swapEmails(int first, int second) {
    list<Email>::iterator firstIt = getIterator(first);
    list<Email>::iterator secondIt = getIterator(second);

    Email temp = *firstIt;
    *firstIt = *secondIt;
    *secondIt = temp;
}

// Move a newly inserted email up the heap
void heapifyUp(int index) {

    while (index > 0) {

        int parent = (index - 1) / 2;

        list<Email>::iterator childIt = getIterator(index);
        list<Email>::iterator parentIt = getIterator(parent);

        if (childIt->hasHigherPriorityThan(*parentIt)) {
            swapEmails(index, parent);
            index = parent;
        }
        else {
            break;
        }
    }
}

// Move an email down the heap after deleting the maximum
void heapifyDown(int index) {

    int size = heap.size();

    while (true) {

        int leftChild = 2 * index + 1;
        int rightChild = 2 * index + 2;
```

```
            int highest = index;

            if (leftChild < size) {

                list<Email>::iterator leftIt = getIterator(leftChild);
                list<Email>::iterator highestIt = getIterator(highest);

                if (leftIt->hasHigherPriorityThan(*highestIt)) {
                    highest = leftChild;
                }
            }

            if (rightChild < size) {

                list<Email>::iterator rightIt = getIterator(rightChild);
                list<Email>::iterator highestIt = getIterator(highest);

                if (rightIt->hasHigherPriorityThan(*highestIt)) {
                    highest = rightChild;
                }
            }

            if (highest != index) {
                swapEmails(index, highest);
                index = highest;
            }
            else {
                break;
            }
        }
    }

public:

    MaxHeap() {
    }

    // Add an email to the priority queue
    void insert(const Email& email) {
```

```
    heap.push_back(email);

    int newIndex = heap.size() - 1;

    heapifyUp(newIndex);
}

// Return the highest priority email without deleting it
Email* getMax() {

    if (heap.empty()) {
        return nullptr;
    }

    return &heap.front();
}

// Delete the highest priority email
void removeMax() {

    if (heap.empty()) {
        return;
    }

    // Only one email
    if (heap.size() == 1) {
        heap.pop_front();
        return;
    }

    // Move last email to the root
    heap.front() = heap.back();

    // Delete last email
    heap.pop_back();

    // Restore heap
    heapifyDown(0);
}
```

```
    int getCount() const {
        return heap.size();
    }

    bool isEmpty() const {
        return heap.empty();
    }
};


// ============================================================
// EMAIL PRIORITY SYSTEM OBJECT
// ============================================================

class EmailPrioritySystem {
private:
    MaxHeap inbox;

    // Remove spaces from beginning and end
    string trim(string text) {

        while (!text.empty() && text.front() == ' ') {
            text.erase(text.begin());
        }

        while (!text.empty() && text.back() == ' ') {
            text.pop_back();
        }

        return text;
    }

    // Process an EMAIL command
    void processEmail(string line) {

        // Remove "EMAIL "
        string information = line.substr(6);

        int firstComma = information.find(',');
        int secondComma = information.find(',', firstComma + 1);
```

```
    string sender = information.substr(0, firstComma);

    string subject = information.substr(
        firstComma + 1,
        secondComma - firstComma - 1
    );

    string date = information.substr(secondComma + 1);

    sender = trim(sender);
    subject = trim(subject);
    date = trim(date);

    Email newEmail(sender, subject, date);

    inbox.insert(newEmail);
}

// Process NEXT command
void processNext() {

    if (inbox.isEmpty()) {
        return;
    }

    Email* nextEmail = inbox.getMax();

    cout << "Next email:" << endl;
    nextEmail->display();
    cout << endl;
}

// Process READ command
void processRead() {

    if (inbox.isEmpty()) {
        return;
    }
```

```
        inbox.removeMax();
    }

    // Process COUNT command
    void processCount() {

        cout << "There are "
             << inbox.getCount()
             << " emails to read."
             << endl
             << endl;
    }

public:

    EmailPrioritySystem() {
    }

    void processCommand(string line) {

        // Remove Windows carriage return if necessary
        if (!line.empty() && line.back() == '\r') {
            line.pop_back();
        }

        if (line.substr(0, 5) == "EMAIL") {
            processEmail(line);
        }
        else if (line == "NEXT") {
            processNext();
        }
        else if (line == "READ") {
            processRead();
        }
        else if (line == "COUNT") {
            processCount();
        }
    }

    void processFile(string filename) {
```

```
        ifstream inputFile(filename);

        if (!inputFile.is_open()) {
            cout << "Unable to open file." << endl;
            return;
        }

        string line;

        while (getline(inputFile, line)) {

            if (!line.empty()) {
                processCommand(line);
            }
        }

        inputFile.close();
    }
};


// ============================================================
// MAIN
// ============================================================

int main(int argc, char* argv[]) {

    EmailPrioritySystem system;

    // If filename is supplied through command line
    if (argc > 1) {

        system.processFile(argv[1]);
    }

    // Otherwise read commands from standard input
    else {

        string line;
```

```
        while (getline(cin, line)) {

            if (!line.empty()) {
                system.processCommand(line);
            }
        }
    }

    return 0;
}
```

# Gemini Generated Code:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

// Represents an individual Email object
class Email {
private:
    int id;
    int priority;
    std::string sender;
    std::string subject;

public:
    Email() : id(0), priority(0), sender(""), subject("") {}

    Email(int id, int priority, const std::string& sender, const std::string& subject)
        : id(id), priority(priority), sender(sender), subject(subject) {}

    int getPriority() const { return priority; }
    int getId() const { return id; }
    std::string getSender() const { return sender; }
    std::string getSubject() const { return subject; }

    void display() const {
```

```
        std::cout << "Email ID: " << id << " | Priority: " << priority
                  << " | From: " << sender << " | Subject: " << subject << "\n";
    }
};

// Custom Max-Heap implemented using a dynamic list (std::vector)
class MaxHeap {
private:
    std::vector<Email> heap;

    int parent(int i) const { return (i - 1) / 2; }
    int leftChild(int i) const { return (2 * i) + 1; }
    int rightChild(int i) const { return (2 * i) + 2; }

    void heapifyUp(int index) {
        while (index > 0 && heap[parent(index)].getPriority() < heap[index].getPriority()) {
            std::swap(heap[parent(index)], heap[index]);
            index = parent(index);
        }
    }

    void heapifyDown(int index) {
        int maxIndex = index;
        int left = leftChild(index);
        int right = rightChild(index);

        if (left < heap.size() && heap[left].getPriority() > heap[maxIndex].getPriority()) {
            maxIndex = left;
        }
        if (right < heap.size() && heap[right].getPriority() > heap[maxIndex].getPriority()) {
            maxIndex = right;
        }

        if (index != maxIndex) {
            std::swap(heap[index], heap[maxIndex]);
            heapifyDown(maxIndex);
        }
    }

public:
```

```
    bool isEmpty() const {
        return heap.empty();
    }

    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    Email peek() const {
        if (isEmpty()) {
            throw std::runtime_error("Heap is empty");
        }
        return heap[0];
    }

    void extractMax() {
        if (isEmpty()) return;
        heap[0] = heap.back();
        heap.pop_back();
        if (!heap.empty()) {
            heapifyDown(0);
        }
    }
};

// Manager class handling CEO interactions and commands
class EmailManager {
private:
    MaxHeap inbox;

public:
    void loadInbox(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            std::cerr << "Error: Could not open file " << filename << "\n";
            return;
        }

        std::string line;
```

```
    // Parse current inbox line items until commands section starts
    while (std::getline(file, line)) {
        if (line.empty()) continue;

        // If the line matches expected command words, break to delegate execution
        if (line == "NEXT" || line == "READ") {
            processCommand(line);
            break;
        }

        std::stringstream ss(line);
        int id, priority;
        std::string sender, subject;

        if (ss >> id >> priority) {
            ss.ignore(); // skip space
            std::getline(ss, sender, ',');
            std::getline(ss, subject);

            // Trim trailing and leading whitespace modifications if necessary
            inbox.insert(Email(id, priority, sender, subject));
        }
    }

    // Process remaining commands inside the file loop
    while (std::getline(file, line)) {
        if (!line.empty()) {
            processCommand(line);
        }
    }
    file.close();
}

void processCommand(const std::string& command) {
    if (command == "NEXT") {
        if (inbox.isEmpty()) {
            std::cout << "No emails in inbox.\n";
        } else {
            std::cout << "Next urgent email details:\n";
            inbox.peek().display();
```

```
            }
        }
        else if (command == "READ") {
            if (inbox.isEmpty()) {
                std::cout << "No emails to read.\n";
            } else {
                inbox.extractMax();
            }
        }
    }
};

int main() {
    EmailManager manager;
    manager.loadInbox("input.txt");
    return 0;
}
```

# Code Analysis:

**Correctness**: The ChatGPT generated program is much more correct than the Gemini program. ChatGPT correctly reads the required EMAIL, NEXT, READ, and COUNT commands. It also gives each sender category the correct priority and uses the newest date when two emails have the same sender priority. Additionally, it correctly handles repeated NEXT commands without removing the email, repeated READ commands, and NEXT or READ when the heap is empty. Its output matches the required sample format, except that the sender, subject, and date are not indented.

The Gemini generated program does not correctly implement the assignment requirements. It expects each email to contain an ID and numerical priority instead of the required EMAIL <sender category>,<subject>,<date> format. Its Email object does not store the required date, and its display method prints an ID and numerical priority instead of the required sender, subject, and date. It also does not implement the COUNT command. Because of these problems, the Gemini program would not correctly execute the provided sample test file. I

I am very confused as to why the Gemini solution is so different from the ChatGPT version. I have used both of these models for all of my past assignments and each time the outcomes have been different, but similar. This is the first time where a program has completely derailed from the instructions I provided.

**Time Complexity**: The Gemini MaxHeap does have better execution time because it stores the heap in a vector, which allows elements to be accessed directly by index. Insertion takes O(log n), removal of the highest priority email takes O(log n), NEXT takes O(1), and checking whether the heap is empty takes O(1).

The ChatGPT program uses a std::list. Although its heap still has O(log n) levels, the program repeatedly calls getIterator() to reach a specific heap index. Accessing an index in a linked list takes O(n), so the heap operations can take approximately O(n log n) instead of O(log n). NEXT and COUNT are still O(1).

**Space Complexity**: Both programs use O(n) space because the amount of memory used grows with the number of emails stored in the heap.

**Maintainability**: The ChatGPT program is more maintainable because it separates the program into three clear classes: Email, MaxHeap, and EmailPrioritySystem. Each class has one main responsibility. The Email class stores and compares emails, the MaxHeap class manages the priority queue, and the EmailPrioritySystem class processes commands and input. This makes the program easier to understand and modify.

**Program Chosen for Assignment:** I am choosing the ChatGPT program as the basis for my assignment. The main reason is correctness. It already implements the required email format, sender priorities, date comparson, EMAIL, NEXT, READ, and COUNT commands, and produces the required output. It also separates the program into objects with clear responsibilities.

# Improvements:

I am going to change the MaxHeap storage from a list to a vector. This will allow the heap to directly access a parent or child using an index instead of going through a linked list. As a result, insertion and removal will improve from O(n log n) to O(log n).

For correctness, I will change the command detection so that an email command must actually begin with "EMAIL " instead of accepting any line whose first five characters are "EMAIL". I will also add checks for missing commas when parsing an email and improve the trim method so it removes other whitespace such as tabs in addition to spaces. Additionally, I am going to add tabs before the sender, subject, and date to match the sample output.

For maintainability, I will remove the getIterator() method because it is no longer needed with a vector, simplify the heapify methods using direct indexing, and add more detailed comments explaining the major sections of the program.